SIDE A · QUIZ + PRACTICE

CoA Midterm (UTS) · S1–S7 · each question shows its source box on the back as T#

QUIZ (prof's solutions, may be recycled)

Q1 32-bit 0x778899AA → keep the 2 middle bytes. 16-bit hex and binary? T1

Bytes 77 | 88 | 99 | AA → drop MSB 77 and LSB AA → 8899 → 8=1000 8=1000 9=1001 9=1001

0x8899 = 1000 1000 1001 1001

Q2 "ME" = 0x4D45. 16-bit binary? Read as two's complement: + or −? T1 T2

4=0100 D=1101 4=0100 5=0101. MSB (leftmost bit) = 0 → positive.

0100 1101 0100 0101 · positive (+19781)

Q3 200 s now; performance doubles every 2 yrs. Time on a CPU 4 yrs older? T3

4 ÷ 2 = 2 doublings → 2^2 = 4× slower → 200 × 4

800 s

Q4 MARIE: PC=150, M[150]=4155, M[155]=0010, AC=0035. Decode · fetch · execute + final AC T4 T5

Decode: opcode 4 = Subt, address 155 → Subt 155
Fetch: MAR←PC · MBR←M[MAR] · IR←MBR · PC←PC+1 (PC = 151)
Execute: MAR←IR[11–0] · MBR←M[MAR] (0010) · AC←AC−MBR = 0035 − 0010

AC = 0025 hex

Q5 100 Start, LOAD P · ADD Q · SUBT R · STORE S · OUTPUT · HALT · 106 P DEC 60 · 107 Q DEC 40 · 108 R DEC 15 · 109 S HEX 0000. Hex 100–105, symbol table, final AC T4 T6

Instr	LOAD P	ADD Q	SUBT R	STORE S	OUTPUT	HALT
Hex	**1106**	**3107**	**4108**	**2109**	**6000**	**7000**

Symbols: Start 100 · P 106 · Q 107 · R 108 · S 109. Trace: 60 → 100 → 85 → M[109]=85 → out 85

AC = 85 decimal (0055 hex)

Q6 16-bit instr, expanding opcodes: 5 × three 4-bit operands, 30 × two 5-bit, 50 × one 8-bit. Possible? T7

$5\cdot2^{12} + 30\cdot2^{10} + 50\cdot2^8$ = 20,480 + 30,720 + 12,800 = 64,000 ≤ 2^{16} = 65,536

Possible (1,536 patterns unused)

Q7 5-stage pipeline, 40 ns clock, non-pipelined = sum of stages. Speedup for 800 instr? T8

t_n = 5·40 = 200 ns. Non-pipe 800·200 = 160,000 ns. Pipe (5+800−1)·40 = 32,160 ns

S = 160,000 ÷ 32,160 ≈ 4.975

Q8 2^{24} B byte-addr memory, direct-mapped, 1024 blocks × 16 B. # blocks · Tag/Index/Offset · index of 0x5A3C8F T10 T1

Blocks $2^{24}/2^4 = 2^{20}$. Offset $\log_2 16$ = 4 · Index $\log_2 1024$ = 10 · Tag 24−10−4 = 10
0101 1010 00 | 11 1100 1000 | 1111 → index = 512+256+128+64+8

2^{20} blocks · 10 | 10 | 4 · block 968

PRACTICE (made up, same patterns as the quiz)

P1 32-bit 0xC0FFEE42 → 16-bit code from its MSB followed by its LSB. Hex and binary? T1

Bytes C0 | FF | EE | 42 → C0 then 42 → C=1100 0=0000 4=0100 2=0010

0xC042 = 1100 0000 0100 0010

P2 (a) 16-bit 0xF3A0 as two's complement: decimal? (b) −37 in 8-bit two's complement? T2

(a) F ≥ 8 → negative. Unsigned 62,368 − 2^{16} (65,536)
(b) +37 = 00100101 → flip 11011010 → +1

(a) −3168 · (b) 11011011 = 0xDB

P3 60 s now; performance doubles every 1.5 yrs. Time on a CPU 6 yrs newer? T3

6 ÷ 1.5 = 4 doublings → 2^4 = 16× faster → 60 ÷ 16

3.75 s

P4 MARIE: PC=200, M[200]=320A, M[20A]=0007, AC=0012 (hex). Decode · fetch · execute · final AC and PC T4 T5

Decode: opcode 3 = Add, address 20A → Add 20A
Fetch: MAR←PC (200) · MBR←M[MAR] (320A) · IR←MBR · PC←PC+1 (201)
Execute: MAR←IR[11–0] (20A) · MBR←M[MAR] (0007) · AC←AC+MBR = 0012 + 0007

AC = 0019 hex · PC = 201

P5 100 LOAD A · 101 SUBT B · 102 SKIPCOND 000 · 103 JUMP UseA · 104 LOAD B · 105 JUMP Done · 106 UseA, LOAD A · 107 Done, STORE M · 108 HALT · 109 A DEC 9 · 10A B DEC 14 · 10B M DEC 0. Hex 100–108, symbol table, final AC and M. What does it compute? T4 T6

Addr	100	101	102	103	104	105	106	107	108
Hex	**1109**	**410A**	**8000**	**9106**	**110A**	**9107**	**1109**	**210B**	**7000**

Symbols: UseA 106 · Done 107 · A 109 · B 10A · M 10B
Trace: AC 9 → 9 − 14 = −5 → AC < 0 so skip 103 → LOAD B (14) → JUMP Done → STORE M

AC = 14 = M · stores max(A, B)

P6 12-bit instr, 4-bit address fields: 14 two-address, 32 one-address, 16 zero-address. Possible? T7

$14\cdot2^8 + 32\cdot2^4 + 16\cdot2^0$ = 3584 + 512 + 16 = 4112 > 2^{12} = 4096
Escape check: 16 − 14 = 2 left → $2\cdot2^4$ = 32, all used → 0 left for zero-address

Not possible

P7 4-stage pipeline, 25 ns clock, 100 instr, non-pipelined instr = 100 ns. Speedup? With 6 stall cycles? T8

Non-pipe 100·100 = 10,000 ns. Pipe (4+100−1)·25 = 2575 ns. With stalls (103+6)·25 = 2725 ns

S ≈ 3.883 · with stalls ≈ 3.670

P8 2^{16} B byte-addr memory, 64-block cache, 8 B/block. # blocks · direct-mapped fields + line of 0xBEEF · 2-way fields + set of 0xBEEF T10 T1

Blocks $2^{16}/2^3 = 2^{13}$. Direct: offset 3 · index $\log_2 64$ = 6 · tag 16−6−3 = 7
0xBEEF = 1011111 | 011101 | 111 → line 29 (tag 95)
2-way: 64/2 = 32 sets → offset 3 · set 5 · tag 8 → 10111110 | 11101 | 111 → set 29 (tag 190)

2^{13} blocks · 7|6|3, line 29 · 8|5|3, set 29

P9 8M × 32 memory. Address bits if word-addressable? Byte-addressable? T9

Word: 8M = 2^{23} units. Byte: 8M × (32/8) = 32M = 2^{25} units

23 bits · 25 bits

P10 (a) Store 0xDEADBEEF at address 200, big and little endian. (b) Little-endian memory holds 78 56 34 12 at 0–3. Value? T12

Addr	200	201	202	203
Big	DE	AD	BE	EF
Little	EF	BE	AD	DE

(b) Lowest address holds the LSB → reverse the bytes: 12 34 56 78

(a) table above · (b) 0x12345678

P11 (a) Direct-mapped, 4 blocks, refs 2, 3, 6, 2, 10, 3, 6: hit/miss? (b) Fully associative, 2 blocks, refs 1, 2, 1, 3, 1, 2: LRU vs FIFO hits? T11

(a) line = block mod 4 → 2,3,2,2,2,3,2. Line 2 keeps being replaced: M M M M M **H** M
(b) LRU: M M H · 3 evicts 2 (1 was just used) · 1 H · 2 M
FIFO: M M H · 3 evicts 1 (loaded first) · 1 M · 2 M

(a) 1 hit · (b) LRU 2 hits, FIFO 1 hit

P12 LOAD 300. M[300]=500, M[500]=700, M[800]=900, index R1=500. Value loaded for each mode? T13

Immediate: 300 itself · Direct: M[300] · Indirect: M[M[300]] = M[500] · Indexed: EA = 300+500 = 800 → M[800]

300 · 500 · 700 · 900

P13 4×10^8 instr, CPI 2.5, 2 GHz. New CPU: CPI 3, 2.5 GHz. CPU times and speedup? T3

Old 4e8 × 2.5 ÷ 2e9 = 0.5 s · New 4e8 × 3 ÷ 2.5e9 = 0.48 s · S = 0.5 ÷ 0.48

0.5 s · 0.48 s · S ≈ 1.042 (new is faster)

P14 (a) (A+B)×(C−D) to postfix. (b) Z = (A+B)×C on a 3-address and a 1-address machine T14

(a) Write operands, then the operator: A B + · C D − · ×
(b) 3-addr: ADD R1,A,B · MULT Z,R1,C
1-addr (AC): LOAD A · ADD B · MULT C · STORE Z

(a) A B + C D − ×

SIDE B · THEORY & FORMULAS look up the T# from the front

T1 Hex ↔ binary · bytes

0 0000	4 0100	8 1000	C 1100
1 0001	5 0101	9 1001	D 1101
2 0010	6 0110	A 1010	E 1110
3 0011	7 0111	B 1011	F 1111

1 hex digit = 4 bits · 1 byte = 2 hex digits. 32-bit = 4 bytes: **MSB** = leftmost pair, **LSB** = rightmost. Bin → hex: group 4 from the right. Bits → decimal: Σ bit × 2^position (…128 64 32 16 8 4 2 1). ASCII: 7 bits in 1 byte ('A' 41, 'a' 61, '0' 30); same bits can be text, number or instruction.

T2 Two's complement (n bits)

Range -2^{n-1} … $2^{n-1} - 1$ (8-bit −128…127 · 16-bit −32768…32767). Unsigned: 0 … $2^n - 1$.
Sign: MSB 0 → +, MSB 1 → −. Hex: first digit 0–7 → +, 8–F → −.
Negate: write +value, flip all bits, add 1 (−10: 00001010 → 11110101 → 11110110).
Decode a negative: value = unsigned − 2^n (0xF6 = 246 − 256 = −10).

T3 Moore's Law · CPU performance

Factor = 2^(years ÷ doubling period). Time ∝ 1/performance: older CPU → time × factor; newer → time ÷ factor.
CPU time = IC × CPI ÷ f = IC × CPI × T · cycles = IC × CPI · f = 1/T (1 GHz ↔ 1 ns, 2 GHz ↔ 0.5 ns) · **Speedup = old time ÷ new time**.

T4 MARIE: format, registers, opcodes

4K words = 2^{12} → **12-bit address**; 16-bit word/instr: **opcode IR[15–12] | address IR[11–0]** → 1 hex digit + 3 hex digits. Word-addressable, accumulator-based.
AC 16 (data) · MAR 12 (address) · MBR 16 (memory data) · PC 12 (next instr) · IR 16 (current instr) · InREG/OutREG 8.

Op	Instr	Meaning	Op	Instr	Meaning
1	Load X	AC ← M[X]	6	Output	OutREG ← AC
2	Store X	M[X] ← AC	7	Halt	stop
3	Add X	AC ← AC + M[X]	8	Skipcond	skip next if 000: AC<0 · 400: AC=0 · 800: AC>0
4	Subt X	AC ← AC − M[X]	9	Jump X	PC ← X
5	Input	AC ← InREG			

T5 Fetch-decode-execute (RTN)

Fetch	MAR ← PC · MBR ← M[MAR] · IR ← MBR · PC ← PC + 1
Decode	opcode = IR[15–12] · MAR ← IR[11–0]
Load	MBR ← M[MAR] · AC ← MBR
Store	MBR ← AC · M[MAR] ← MBR
Add / Subt	MBR ← M[MAR] · AC ← AC ± MBR

Memory is always reached through MAR (address) and MBR (data). PC already points to the next instruction during execute.

T6 Assembling a MARIE program

1. **Symbol table** (pass 1): every label → its address. 2. **Hex** (pass 2): opcode digit + 3-digit address of the operand label. No operand (Input, Output, Halt) → 000. Skipcond keeps its condition (8000/8400/8800). DEC/HEX lines are data (DEC 60 = 003C).
3. **Trace**: track AC, PC and memory written by Store. Jump/Skipcond change PC.

T7 Expanding opcodes

Pattern count (prof's method): each instruction uses 2^(operand bits) patterns.
Σ count × 2^(operand bits) ≤ 2^(instruction bits) → possible.
Escape check: opcode bits = instr bits − operand bits. Group 1 gets 2^(opcode bits) codes; leftover codes × 2^(extra opcode bits) feed the next group. Leftover 0 too early → impossible.

T8 Pipelining

k stages, n instr, τ cycle, t_n non-pipelined instr time (= kτ if equal stages).
Pipelined = (k + n − 1)·τ (+ s stalls: (k + n − 1 + s)·τ) · **Non-pipelined = n·t_n** · **S = n·t_n ÷ ((k+n−1)·τ)** → k as n grows.
Hazards: structural (same hardware) · data (result not ready → forwarding) · control (branch → flush/prediction).

T9 Address bits

bits = log₂(# addressable units). Word-addressable: units = # words. Byte-addressable: units = words × (bits per word ÷ 8).
2^{10} 1K · 2^{12} 4K · 2^{16} 64K · 2^{20} 1M · 2^{22} 4M · 2^{23} 8M · 2^{24} 16M · 2^{25} 32M · 2^{30} 1G

T10 Cache address fields

Address = **Tag | Index (or Set) | Offset**, split from the right.
Offset = log₂(block bytes) · Index = log₂(# lines) · Set bits = log₂(lines ÷ N) for N-way · Fully associative: no index · Tag = remaining bits.
# memory blocks = memory ÷ block size · block # = ⌊addr ÷ block⌋ · line = block mod lines · set = block mod sets.

T11 Cache tracing · replacement · writes

Direct: line = block mod lines; new block always replaces the old one. **Set assoc.:** set = block mod sets, any line in the set. **Fully:** any line.
Hit = block already there. Hit rate = hits ÷ accesses.
LRU: evict least recently used (a hit refreshes it). **FIFO**: evict oldest loaded (hits change nothing). LFU: fewest uses. Same result unless a hit happens before an eviction.
Write-through: cache + memory every write (consistent, more traffic). **Write-back**: cache only + dirty bit, memory written on replacement.

T12 Endianness

Big: MSB at lowest address (write bytes left → right). **Little**: LSB at lowest address (reverse the bytes). Pad to full width first (0x1234 as 32-bit = 00 00 12 34). Bits inside a byte never change.
Big: UNIX, MIPS, network order. Little: Intel x86. Bi-endian: MIPS, ARM.

T13 Addressing modes

Immediate: operand in the instruction · Direct: EA = A → M[A] · Register: in a register · Indirect: EA = M[A] → M[M[A]] · Indexed: EA = A + index reg → M[EA] · Array: base + i × element size.

T14 Instruction formats · postfix

Postfix (RPN): operands then operator; (A+B)×C → A B + C ×. Stack (0-addr) · Accumulator (1-addr, MARIE) · GPR (2/3-addr).
3-addr: OP dest,src1,src2 · 2-addr: OP dest,src (dest = dest OP src) · 1-addr: OP X uses AC. Fixed length = fast decode; variable = compact.

C Concepts for short-answer questions

- **Architecture** = what the programmer sees (ISA, registers, data types, addressing). **Organization** = how it's built (control signals, buses, memory tech).
- **Von Neumann**: stored program, CPU + memory + I/O, one shared CPU–memory path = **bottleneck**. Harvard: separate instruction/data paths.
- **Bus**: data · address · control lines. **CPU** = datapath (registers, ALU) + control unit. Hardwired control fast/rigid; microprogrammed flexible/slower.
- **CISC**: many complex variable-length instr (x86). **RISC**: few simple fixed-length, load/store (MIPS, ARM).
- **Hierarchy** registers → cache → RAM → disk: capacity ↑, cost/bit ↓, latency ↑ going down. Goal: near-fastest speed at near-cheapest cost.
- **SRAM** flip-flop, fast, cache. **DRAM** capacitor, refresh, main memory. **Temporal** locality: reuse soon; **spatial**: neighbours soon.
- **Virtual memory**: page table/MMU maps virtual → physical; page fault = page on disk; TLB caches page-table entries.
- 2 driving factors of computer development: cost and performance. Moore's limits: atom size, heat, cost.